Parcial 4 – Modelo de Práctica (Seguros de vivienda)

Enunciado

Una empresa que vende seguros para la vivienda necesita organizar y analizar la información de los seguros que vendió. De cada Seguro se conoce: código de producto (entero positivo), nombre del cliente (cadena de caracteres), tipo de vivienda (entero entre 1 y 5, donde: 1-Casa, 2-Departamento, 3-Dúplex, 4-PH, 5-Cabaña), si el asegurado es el dueño de la vivienda (booleano: True si es propietario, False si no lo es) y monto mensual que paga por el seguro (número flotante positivo).

En base a lo anterior, se provee ya desarrollado:

- Un módulo `clases.py` que contiene la definición de la clase `Seguro` con el método constructor y el método `__str__()` ya incluidos. **NO CAMBIE NADA. NO AGREGUE NADA. DEJE LA CLASE TAL CUAL ESTÁ.**
- Un segundo módulo `principal.py` que contiene una función `main()` con el esquema del menú de opciones ya desarrollado y funcionando, de forma que esa función `main()` sea usada como la función de entrada del programa completo, y una función de validación de carga. En el mismo módulo, se provee ya desarrollada una función `generar_aleatorio()` que crea con valores aleatorios un objeto de la clase `Seguro` y lo retorna ya listo para usar.

Los procesos para cumplir con cada opción del menú son justamente la tarea que debe programar cada estudiante. Las opciones a programar son:

Opciones del menú

1. Cargar un arreglo de registros con los datos de n seguros (cargar n por teclado y validar que sea positivo), de manera que en todo momento el arreglo se mantenga ordenado por nombre del cliente. Para esto debe utilizar el algoritmo de inserción ordenada con búsqueda binaria (se considerará directamente incorrecta la solución basada en cargar el arreglo completo y ordenarlo al final, o aplicar el algoritmo de inserción ordenada pero con búsqueda secuencial). La carga de los seguros debe ser automática y con base aleatoria (use la función `generar_aleatorio()` que ya tiene disponible en el módulo `principal.py` para crear los objetos). Cada vez que se ingrese en esta opción, el arreglo debe ser creado desde cero y todo contenido anterior debe ser eliminado.
   Ajuste de runner: al ejecutar esta opción, solo cargue por teclado el valor de n, y no muestre nada al terminar de cargar el vector. Solo regrese al menú. Si el nombre de un cliente ya existe en el arreglo, el nuevo seguro debe insertarse a continuación de los que ya tienen ese mismo nombre (así todos los programas generan el mismo orden).
2. Mostrar todos los seguros del arreglo, a razón de un registro por línea, de forma tal que en lugar del valor booleano que indica si es dueño de la vivienda se vea la cadena "Propietario" o "No propietario" según corresponda. Determinar el monto promedio pagado para los seguros del tipo de vivienda t, siendo t un valor que se carga por teclado.
   Ajuste de runner: al ejecutar esta opción, cargue por teclado t, muestre el listado pedido usando el método `__str__()` que ya tiene en la clase (que ya hace el reemplazo indicado), muestre al

final el promedio pedido aquí (use ":" para separar el mensaje de salida y el valor mostrado) y regrese al menú (no muestre nada más que eso).

3. Buscar si existe un seguro para el cliente cuyo nombre es nom, siendo nom un valor cargado por teclado. Si se le ha vendido un seguro a este cliente, hacerle un 10% de descuento al monto que paga si es propietario, o un 15% de descuento si el cliente no es el propietario de la vivienda. El descuento debe quedar aplicado sobre el monto del seguro guardado en el arreglo.
   Ajuste de runner: al ejecutar esta opción, cargue por teclado el valor de nom. Si el cliente existe, aplique el descuento y muestre el seguro ya con el monto actualizado, en una línea, usando el método __str__(). Si hubiera más de un seguro de ese cliente, el descuento se aplica solo al primero que aparece en el arreglo. Si el cliente no existe, muestre la línea No se encontró ningún seguro para el cliente: seguida del valor de nom. Luego regrese al menú (no muestre nada más que eso).
4. A partir del arreglo de registros, generar un archivo binario donde se incluyan los datos de todos los seguros vendidos para el tipo de vivienda t, siendo este un valor ingresado por teclado.
   Ajuste de runner: al ejecutar esta opción, cargue por teclado el valor de t, genere solo el archivo pedido (si no hay seguros de ese tipo, el archivo queda vacío) y regrese al menú (no muestre nada en la pantalla al terminar).
5. Mostrar el contenido del archivo generado en el punto anterior, a razón de un registro por línea, indicando además una línea extra al final con el monto promedio de los seguros guardados en el archivo. Nota: para el cálculo del promedio, debe recorrer el mismo archivo, Y NO calcularlo desde el vector.
   Ajuste de runner: al ejecutar esta opción, no cargue nada por teclado, muestre el listado pedido usando el método __str__(), muestre al final el promedio pedido redondeado a dos decimales (use ":" para separar el mensaje de salida y el valor mostrado; si el archivo no tiene registros, el promedio es 0) y regrese al menú (no muestre nada más que eso).

Criterios generales de evaluación

- Estructura completa del programa terminado: máximo 4 puntos (convenciones de estilo, estructura de las funciones que desarrolle, control de ejecución del módulo principal y todo otro elemento que aporte a la estructura del programa).
- Desarrollo correcto del ítem 1: máximo 4 puntos.
- Desarrollo correcto del ítem 2: máximo 4 puntos.
- Desarrollo correcto del ítem 3: máximo 4 puntos.
- Desarrollo correcto del ítem 4: máximo 4 puntos.
- Desarrollo correcto del ítem 5: máximo 4 puntos.
- Para aprobar el parcial (con nota mínima de 4), el alumno debe llegar a un porcentaje de al menos 55% del puntaje máximo (esto es, un poco más de 13 puntos).